CSA4704 - DEEP LEARNING
Lab Experiments

Name: Harini. R
Register no: 192324108

Experiment 6: Gradient Descent, SGD, Momentum and Adam Comparison
Problem Statement: A logistics company trains a delivery-time prediction model. Training is slow, and the engineering lead asks which optimiser converges fastest and most reliably. A delivery dataset with distance, traffic index, weight and time of day as inputs and delivery time as the target. The objective is to implement and compare batch GD, SGD, Momentum and Adam. Use NumPy, TensorFlow, Keras, Matplotlib.

Tasks:
1. Implement batch GD, SGD and Momentum from scratch on a simple loss surface.
2. Plot each optimiser's path on a 2-D loss contour.
3. Train the same Keras regression model with SGD, SGD+Momentum and Adam.
4. Plot loss against epochs for all optimisers.
5. Vary the learning rate and note divergence or slow convergence.

Formulas:

- Batch Gradient Descent (GD): Calculates gradients using the entire training dataset.
- Stochastic Gradient Descent (SGD): Updates parameters using one training example at a time.
- Momentum: Uses a moving accumulation of previous gradients to smooth updates.
- Adam: Adapts the learning rate for each parameter using estimates of gradient averages and squared gradients.

The gradient descent update is

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

where η is the learning rate and L is the loss function.

Code:
Code 1: Implement optimizers from scratch and plot their paths

```
import numpy as np
import matplotlib.pyplot as plt

# Loss surface: f(x,y) = x² + 10y²
```

```
# Its unequal curvature makes optimizer paths easy to compare.
def loss(w):
    return w[0]**2 + 10 * w[1]**2

def grad(w):
    return np.array([2 * w[0], 20 * w[1]])

start = np.array([4.0, 3.0])
lr = 0.08
steps = 50

# Batch GD
def batch_gd(start, lr, steps):
    w = start.copy()
    path = [w.copy()]
    for _ in range(steps):
        w -= lr * grad(w)
        path.append(w.copy())
    return np.array(path)

# SGD: use individual component gradients in alternating order
def sgd(start, lr, steps):
    w = start.copy()
    path = [w.copy()]
    for t in range(steps):
        i = t % 2
        g = np.zeros(2)
        g[i] = [2, 20][i] * w[i]
        w -= lr * g
        path.append(w.copy())
    return np.array(path)

# Momentum
def momentum_gd(start, lr, steps, beta=0.9):
    w = start.copy()
    velocity = np.zeros(2)
    path = [w.copy()]
    for _ in range(steps):
        velocity = beta * velocity + grad(w)
        w -= lr * velocity
```

```
        path.append(w.copy())
    return np.array(path)

# Adam
def adam_gd(start, lr, steps, beta1=0.9, beta2=0.999, eps=1e-8):
    w = start.copy()
    m = np.zeros(2)
    v = np.zeros(2)
    path = [w.copy()]
    for t in range(1, steps + 1):
        g = grad(w)
        m = beta1 * m + (1 - beta1) * g
        v = beta2 * v + (1 - beta2) * (g**2)
        m_hat = m / (1 - beta1**t)
        v_hat = v / (1 - beta2**t)
        w -= lr * m_hat / (np.sqrt(v_hat) + eps)
        path.append(w.copy())
    return np.array(path)

paths = {
    "Batch GD": batch_gd(start, 0.045, steps),
    "SGD": sgd(start, 0.045, steps),
    "Momentum": momentum_gd(start, 0.004, steps),
    "Adam": adam_gd(start, 0.2, steps)
}

# Contour plot
x = np.linspace(-4.5, 4.5, 250)
y = np.linspace(-3.5, 3.5, 250)
X, Y = np.meshgrid(x, y)
Z = X**2 + 10*Y**2

plt.figure(figsize=(9, 7))
plt.contour(X, Y, Z, levels=25)
for name, path in paths.items():
    plt.plot(path[:, 0], path[:, 1], marker=".", markersize=3,
             label=name)
plt.scatter(0, 0, marker="*", s=150, label="Minimum")
plt.xlabel("w1")
plt.ylabel("w2")
```

```
plt.title("Optimizer Paths on a Loss Contour")
plt.legend()
plt.grid(True)
plt.show()

for name, path in paths.items():
    print(name, "final loss =", loss(path[-1]))
```

Code 2: Compare SGD, Momentum and Adam using Keras

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

np.random.seed(42)
tf.keras.utils.set_random_seed(42)

# Synthetic logistics dataset
n = 1200
X = np.column_stack([
    np.random.uniform(1, 50, n),     # Distance
    np.random.uniform(0, 10, n),     # Traffic index
    np.random.uniform(0.5, 30, n),   # Weight
    np.random.uniform(0, 23, n)      # Time of day
])

y = (
    12 + 1.4 * X[:, 0] + 2.5 * X[:, 1]
    + 0.25 * X[:, 2]
    + 3 * np.sin(2 * np.pi * X[:, 3] / 24)
    + np.random.normal(0, 4, n)
)

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)

def make_model():
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(4,)),
        tf.keras.layers.Dense(32, activation="relu"),
        tf.keras.layers.Dense(16, activation="relu"),
        tf.keras.layers.Dense(1)
    ])
    model.compile(loss="mse", optimizer="sgd", metrics=["mae"])
    return model

optimizers = {
    "SGD": tf.keras.optimizers.SGD(learning_rate=0.01),
    "Momentum": tf.keras.optimizers.SGD(
        learning_rate=0.01, momentum=0.9
    ),
    "Adam": tf.keras.optimizers.Adam(learning_rate=0.01)
}

histories = {}

for name, optimizer in optimizers.items():
    tf.keras.utils.set_random_seed(42)
    model = make_model()
    model.compile(optimizer=optimizer, loss="mse", metrics=["mae"])

    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=60,
        batch_size=32,
        verbose=0
    )
    histories[name] = history.history

plt.figure(figsize=(9, 5))
for name, history in histories.items():
    plt.plot(history["val_loss"], label=name)
```

```
plt.xlabel("Epoch")
plt.ylabel("Validation MSE")
plt.title("Optimizer Comparison")
plt.legend()
plt.grid(True)
plt.show()

for name, history in histories.items():
   print(name, "final validation MSE:",
         round(history["val_loss"][-1], 3))
```

Code 3: Investigate learning rates

```
learning_rates = [0.0001, 0.01, 0.1, 1.0]
results = {}

for lr in learning_rates:
   tf.keras.utils.set_random_seed(42)
   model = make_model()
   model.compile(
      optimizer=tf.keras.optimizers.SGD(learning_rate=lr),
      loss="mse"
   )

   history = model.fit(
      X_train, y_train,
      validation_data=(X_val, y_val),
      epochs=30,
      batch_size=32,
      verbose=0
   )
   results[lr] = history.history["val_loss"]

plt.figure(figsize=(9, 5))
for lr, losses in results.items():
   plt.plot(losses, label=f"LR={lr}")

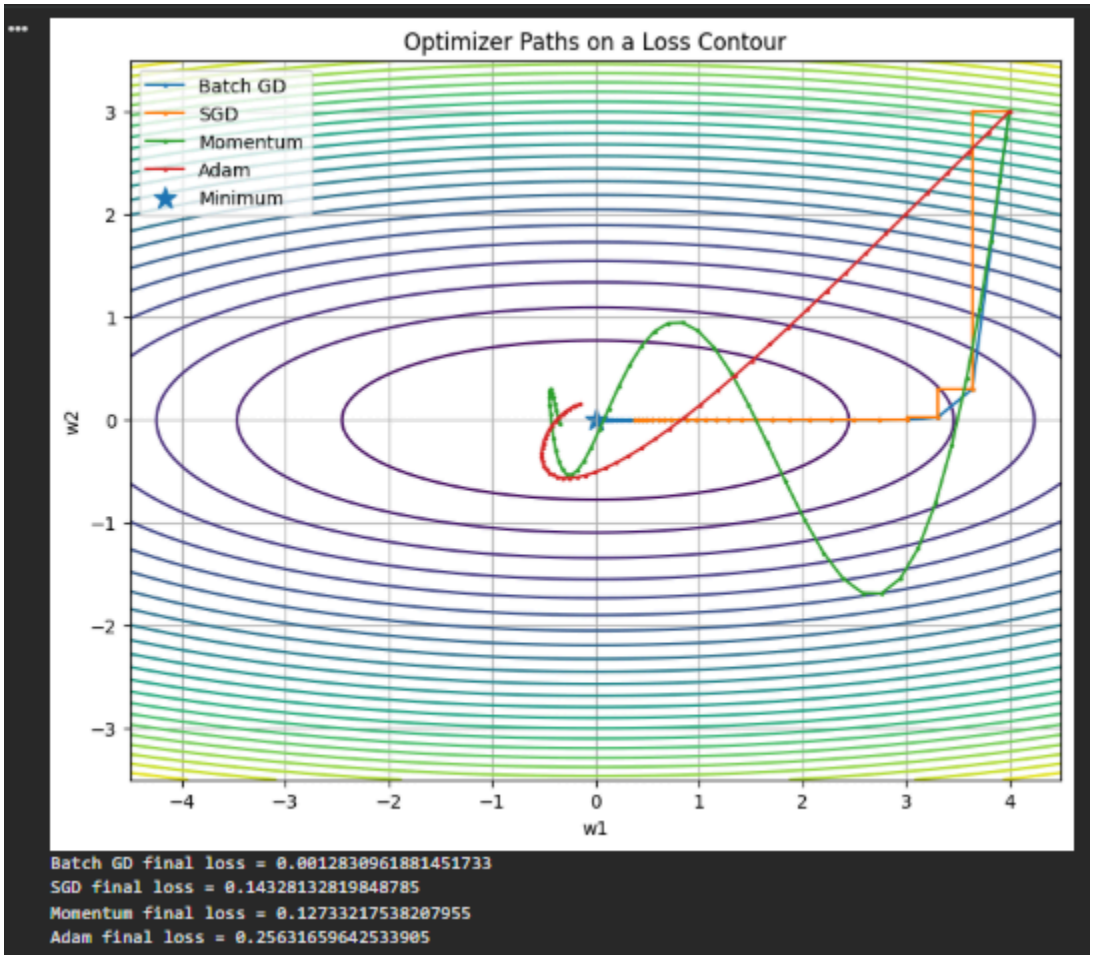
plt.yscale("log")
```

```
plt.xlabel("Epoch")
plt.ylabel("Validation MSE (log scale)")
plt.title("Effect of Learning Rate")
plt.legend()
plt.grid(True)
plt.show()

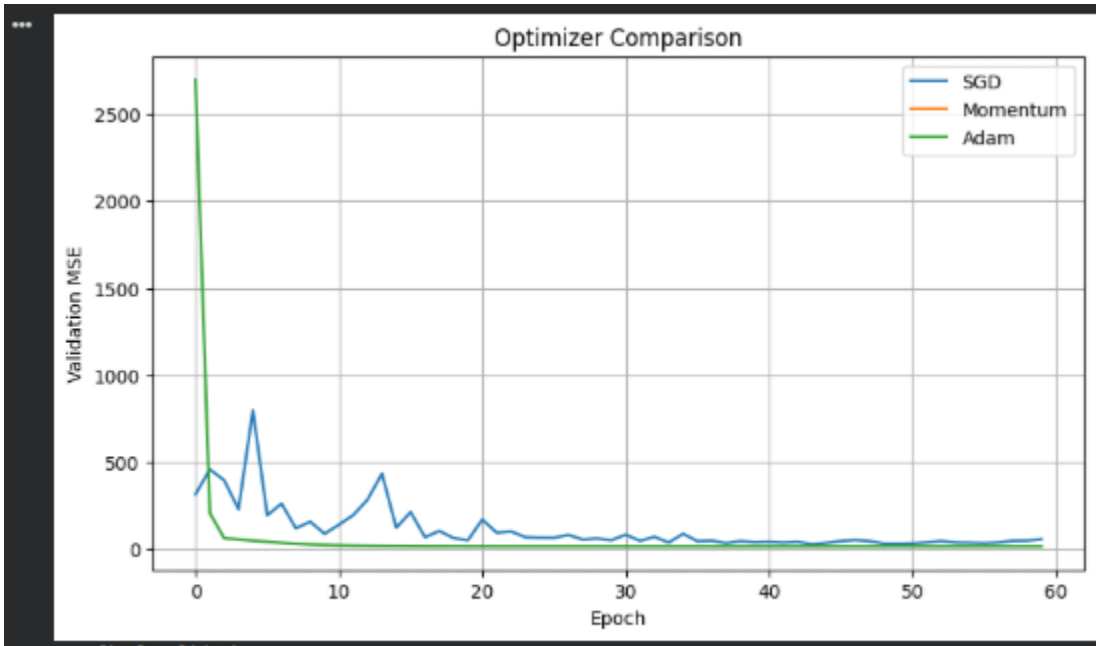
for lr, losses in results.items():
    print(lr, "final validation loss:", losses[-1])
```

Output:

Code 1:

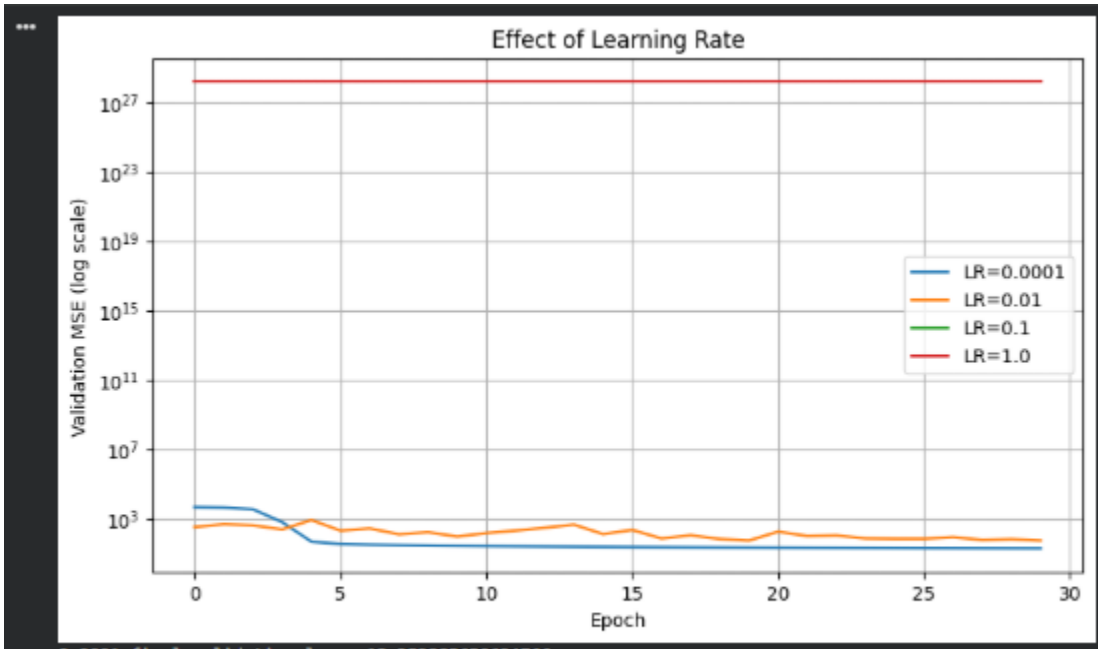


Code 2:




```
SGD final validation MSE: 58.499
Momentum final validation MSE: nan
Adam final validation MSE: 16.463
```

Code 3:




```
0.0001 final validation loss: 18.353885650634766
0.01 final validation loss: 52.38276672363281
0.1 final validation loss: nan
1.0 final validation loss: 1.6254773848680535e+28
```

Experiment 7: Perceptron and Forward Propagation

Problem Statement: A bank wants a quick rule-based screener that approves or rejects small loan applications from two scores. 100 past applications with credit score and income-to-debt ratio, each labelled Approved or Rejected. A second dataset follows the XOR pattern. The objective is to implement a single perceptron with forward propagation and the perceptron learning rule. Use NumPy, Matplotlib.

Tasks:

1. Implement weighted sum, bias and step activation in NumPy.

2. Train the perceptron on the loan data and record weight updates per epoch.

3. Plot the learned decision boundary.

4. Train on the XOR data and show that it fails to converge.

5. Compare step, sigmoid and ReLU outputs for the same inputs.

Formulas:

The weighted sum is

$$z = w^T x + b$$

For a step activation,

$$b \leftarrow b + \eta(y - \hat{y})$$

Code:

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

def step(z):
    return (z >= 0).astype(int)

def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

def relu(z):
    return np.maximum(0, z)

# 1. Generate illustrative loan data
n = 100
```

```
X = np.random.randn(n, 2)

# Synthetic labels based on a linear rule
y = ((1.2 * X[:, 0] + 0.9 * X[:, 1] - 0.2) > 0).astype(int)

# 2. Train a single perceptron
w = np.zeros(2)
b = 0.0
lr = 0.1
epoch_errors = []
weight_history = []

for epoch in range(50):
    errors = 0

    for xi, target in zip(X, y):
        pred = int(step(np.dot(w, xi) + b))
        error = target - pred

        w += lr * error * xi
        b += lr * error
        errors += int(error != 0)

    epoch_errors.append(errors)
    weight_history.append([w[0], w[1], b])

    if errors == 0:
        break

print("Learned weights:", w)
print("Learned bias:", b)
print("Epochs trained:", len(epoch_errors))

# Plot errors per epoch
plt.figure(figsize=(7, 4))
plt.plot(epoch_errors, marker="o")
plt.xlabel("Epoch")
plt.ylabel("Number of misclassified samples")
plt.title("Perceptron Learning")
plt.grid(True)
```

```
plt.show()

# Plot the learned decision boundary
plt.figure(figsize=(7, 6))
plt.scatter(X[y == 0, 0], X[y == 0, 1], label="Rejected")
plt.scatter(X[y == 1, 0], X[y == 1, 1], label="Approved")

x_line = np.linspace(X[:, 0].min(), X[:, 0].max(), 200)
if abs(w[1]) > 1e-10:
    y_line = -(w[0] * x_line + b) / w[1]
    plt.plot(x_line, y_line, "k--", label="Decision boundary")

plt.xlabel("Credit-related score (synthetic)")
plt.ylabel("Income-to-debt-related score (synthetic)")
plt.title("Perceptron Decision Boundary")
plt.legend()
plt.grid(True)
plt.show()

# 3. XOR dataset
X_xor = np.array([
    [0, 0], [0, 1], [1, 0], [1, 1]
])
y_xor = np.array([0, 1, 1, 0])

w_xor = np.zeros(2)
b_xor = 0.0
xor_errors = []

for epoch in range(20):
    errors = 0
    for xi, target in zip(X_xor, y_xor):
        pred = int(step(np.dot(w_xor, xi) + b_xor))
        error = target - pred
        w_xor += 0.1 * error * xi
        b_xor += 0.1 * error
        errors += int(error != 0)
    xor_errors.append(errors)

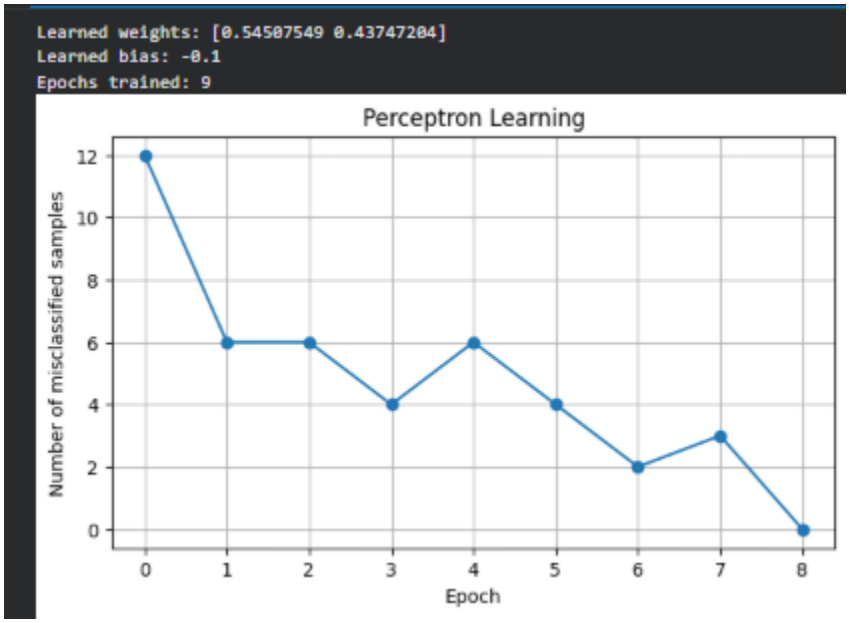
print("\nXOR errors per epoch:", xor_errors)
```

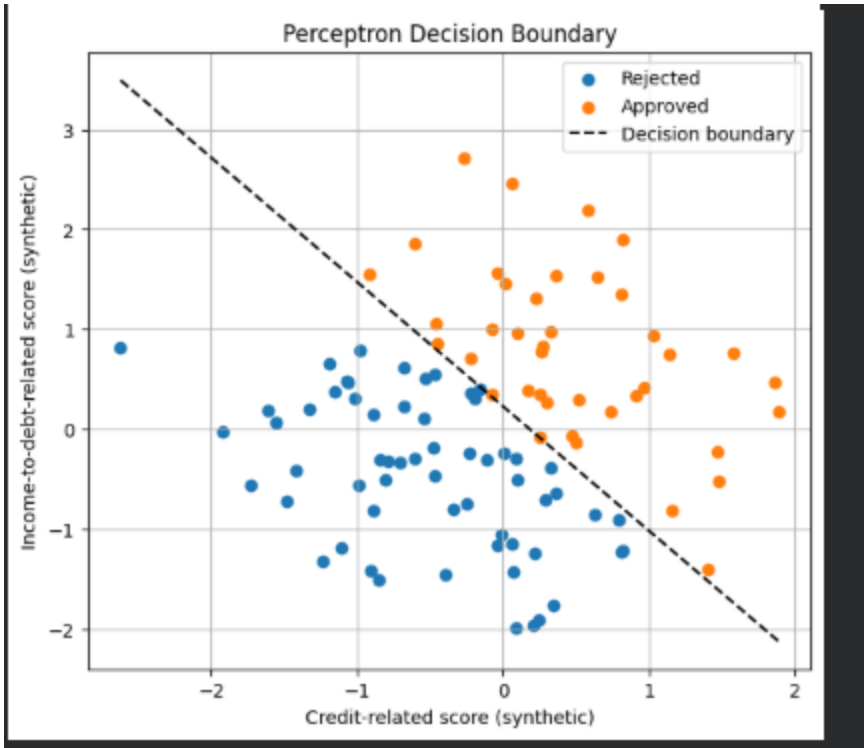
```
print("XOR converged perfectly:",
      np.all(step(X_xor @ w_xor + b_xor) == y_xor))

plt.figure(figsize=(7, 4))
plt.plot(xor_errors, marker="o")
plt.xlabel("Epoch")
plt.ylabel("XOR misclassifications")
plt.title("Single Perceptron on XOR")
plt.grid(True)
plt.show()

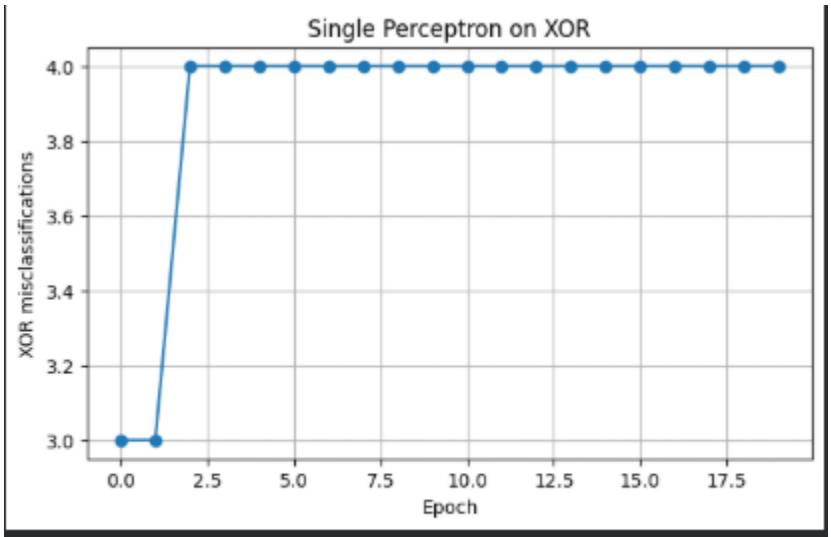
# 4. Compare activation outputs
z = np.array([-2, -1, 0, 1, 2])
print("\nInput:", z)
print("Step:", step(z))
print("Sigmoid:", np.round(sigmoid(z), 3))
print("ReLU:", relu(z))
```

Output:






```
XOR errors per epoch: [3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
XOR converged perfectly: False
```




```
Input: [-2 -1  0  1  2]
Step: [0 0 1 1 1]
Sigmoid: [0.119 0.269 0.5   0.731 0.881]
ReLU: [0 0 0 1 2]
```

Experiment 8: Two-Layer Neural Network with Backpropagation

Problem Statement: The bank's screener from Experiment 7 fails on non-linear cases. The data team needs a hidden layer and must understand how errors flow back. The XOR-style loan dataset and a two- moons customer-segmentation dataset. The objective is to build a

2-layer network from scratch and implement backpropagation manually. Use NumPy, Matplotlib.

Tasks:

1. Initialise weights for an input–hidden–output network in NumPy.
2. Write the forward pass with sigmoid activations.
3. Derive and code the backward pass using the chain rule.
4. Train with gradient descent and plot loss against epochs.
5. Plot the non-linear decision boundary; verify gradients numerically for one weight.

Formulas:

For a network with one hidden layer:

$$H = \sigma(XW_1 + b_1)$$

$$\hat{Y} = \sigma(HW_2 + b_2)$$

For binary cross-entropy, the output-layer error term is

$$\delta_2 = \hat{Y} - Y$$

The hidden-layer error is

$$\delta_1 = (\delta_2 W_2^T) \odot H(1 - H)$$

The gradients are calculated from these error terms and used to update the weights.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons

np.random.seed(42)

# Two-moons nonlinear dataset
X, y = make_moons(n_samples=400, noise=0.15, random_state=42)
```

```
Y = y.reshape(-1, 1)

# Standardize features
X = (X - X.mean(axis=0)) / X.std(axis=0)

# Network dimensions: 2 inputs, 8 hidden neurons, 1 output
input_size = 2
hidden_size = 8
output_size = 1

W1 = np.random.randn(input_size, hidden_size) * 0.5
b1 = np.zeros((1, hidden_size))
W2 = np.random.randn(hidden_size, output_size) * 0.5
b2 = np.zeros((1, output_size))

def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

def forward(X, W1, b1, W2, b2):
    H = sigmoid(X @ W1 + b1)
    O = sigmoid(H @ W2 + b2)
    return H, O

def binary_cross_entropy(Y, O):
    eps = 1e-10
    O = np.clip(O, eps, 1 - eps)
    return -np.mean(Y * np.log(O) + (1-Y) * np.log(1-O))

# 1–4. Forward pass, backpropagation, gradient descent
lr = 0.5
epochs = 5000
losses = []
m = len(X)

for epoch in range(epochs):
    H, O = forward(X, W1, b1, W2, b2)
    loss = binary_cross_entropy(Y, O)
    losses.append(loss)

    # Output delta for sigmoid + binary cross-entropy
```

```
    dZ2 = (O - Y) / m
    dW2 = H.T @ dZ2
    db2 = np.sum(dZ2, axis=0, keepdims=True)

    # Backpropagate through the hidden sigmoid
    dH = dZ2 @ W2.T
    dZ1 = dH * H * (1 - H)
    dW1 = X.T @ dZ1
    db1 = np.sum(dZ1, axis=0, keepdims=True)

    W1 -= lr * dW1
    b1 -= lr * db1
    W2 -= lr * dW2
    b2 -= lr * db2

plt.figure(figsize=(8, 4))
plt.plot(losses)
plt.xlabel("Epoch")
plt.ylabel("Binary cross-entropy")
plt.title("Two-Layer Neural Network Training Loss")
plt.grid(True)
plt.show()

# 5. Plot nonlinear decision boundary
xx, yy = np.meshgrid(
    np.linspace(X[:, 0].min()-0.5, X[:, 0].max()+0.5, 250),
    np.linspace(X[:, 1].min()-0.5, X[:, 1].max()+0.5, 250)
)
grid = np.c_[xx.ravel(), yy.ravel()]
_, grid_pred = forward(grid, W1, b1, W2, b2)

plt.figure(figsize=(7, 6))
plt.contourf(xx, yy, grid_pred.reshape(xx.shape),
             levels=[0, 0.5, 1], alpha=0.3)
plt.contour(xx, yy, grid_pred.reshape(xx.shape),
            levels=[0.5], colors="black")
plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors="k")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Two-Layer Network Decision Boundary")
```

```
plt.show()

# 6. Numerical gradient check for one W1 weight
# Compare analytical derivative with central finite differences.
def full_loss(W1_test):
    _, O_test = forward(X, W1_test, b1, W2, b2)
    return binary_cross_entropy(Y, O_test)

H, O = forward(X, W1, b1, W2, b2)
dZ2 = (O - Y) / m
dH = dZ2 @ W2.T
dZ1 = dH * H * (1 - H)
dW1 = X.T @ dZ1

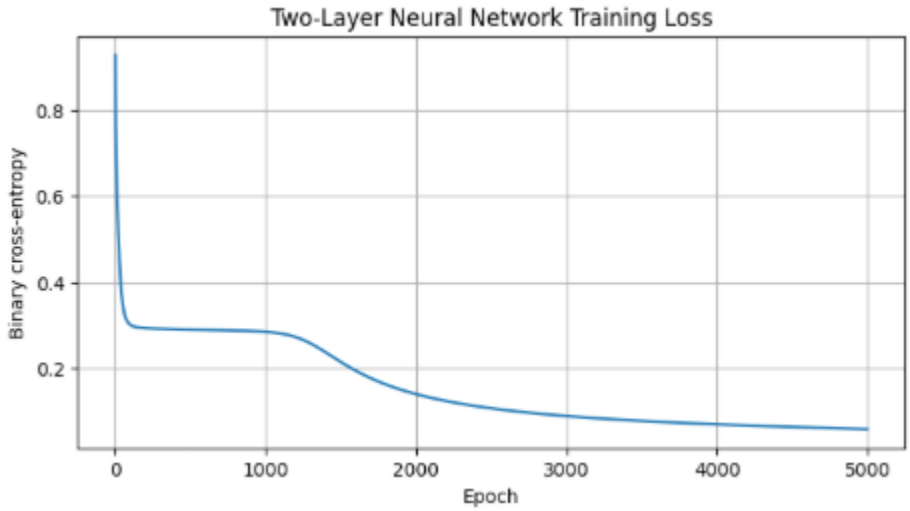
i, j = 0, 0
epsilon = 1e-5

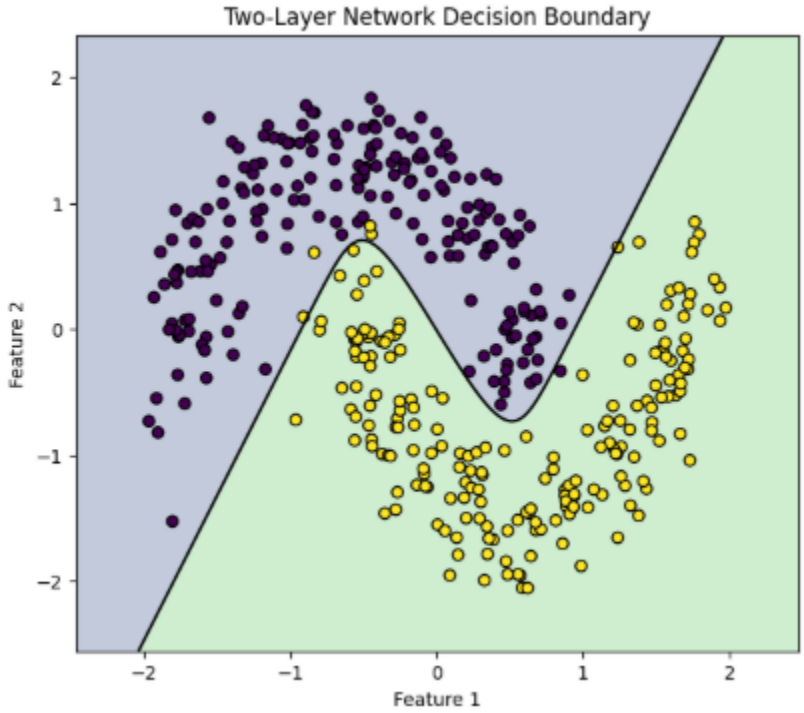
W_plus = W1.copy()
W_minus = W1.copy()
W_plus[i, j] += epsilon
W_minus[i, j] -= epsilon

numerical_grad = (
    full_loss(W_plus) - full_loss(W_minus)
) / (2 * epsilon)

print("Analytical gradient:", dW1[i, j])
print("Numerical gradient:", numerical_grad)
print("Gradient check passed:",
      np.isclose(dW1[i, j], numerical_grad, atol=1e-5))
```

Output:






```
Analytical gradient: 1.9527601117110872e-05
Numerical gradient: 1.952760141088472e-05
Gradient check passed: True
```

Experiment 9: Multilayer Perceptron Implementation

Problem Statement: A telecom operator wants to predict which customers will cancel their subscription next month. A customer-churn dataset with tenure, monthly charges, contract type, complaints and data usage, with a Churn label. The objective is to build, train and evaluate an MLP classifier in Keras. Use TensorFlow, Keras, NumPy, Matplotlib.

Tasks:

1. Pre-process the data: encode categorical fields and scale numeric fields.
2. Build an MLP with two or three hidden layers using ReLU and a sigmoid output.
3. Train with binary cross-entropy and Adam; plot training and validation curves.
4. Report accuracy, precision, recall, F1 and the confusion matrix.
5. Vary the hidden-layer width and note its effect.

Formulas:

A multilayer perceptron consists of an input layer, hidden layers, and an output layer.

- ReLU: Introduces nonlinear behavior in hidden layers.
- Sigmoid: Produces a probability between 0 and 1 for binary classification.
- Binary cross-entropy: Measures the difference between predicted probabilities and binary labels.
- Adam: Adjusts parameter updates using estimates of gradient averages and squared gradients.

The output layer predicts a probability of at least 0.5 is commonly classified as churn, although the threshold can be adjusted for the business objective.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, ConfusionMatrixDisplay
)

np.random.seed(42)
tf.keras.utils.set_random_seed(42)

# 1. Generate illustrative telecom customer data
```

```
n = 1200

df = pd.DataFrame({
    "tenure": np.random.randint(1, 73, n),
    "monthly_charges": np.random.uniform(20, 120, n),
    "contract": np.random.choice(
        ["Month-to-month", "One year", "Two year"], n,
        p=[0.55, 0.25, 0.20]
    ),
    "complaints": np.random.poisson(1.2, n),
    "data_usage": np.random.uniform(1, 100, n)
})

# Synthetic churn probability
logit = (
    1.0
    - 0.035 * df["tenure"].to_numpy()
    + 0.018 * (df["monthly_charges"].to_numpy() - 70)
    + 0.65 * df["complaints"].to_numpy()
    - 0.9 * (df["contract"].to_numpy() == "One year")
    - 1.5 * (df["contract"].to_numpy() == "Two year")
)

prob = 1 / (1 + np.exp(-logit))
df["Churn"] = (np.random.random(n) < prob).astype(int)

print(df.head())
print("\nChurn distribution:\n", df["Churn"].value_counts())

# To use your own CSV instead, replace the generation section with:
# df = pd.read_csv("/content/your_telecom_data.csv")
# Make sure the target column is named "Churn".

X = df.drop(columns="Churn")
y = df["Churn"]

numeric_cols = [
    "tenure", "monthly_charges", "complaints", "data_usage"
]
categorical_cols = ["contract"]
```

```
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50,
    random_state=42, stratify=y_temp
)

# Preprocess: scale numeric columns and one-hot encode categories
preprocessor = ColumnTransformer([
    ("num", StandardScaler(), numeric_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols)
])

X_train = preprocessor.fit_transform(X_train).astype(np.float32)
X_val = preprocessor.transform(X_val).astype(np.float32)
X_test = preprocessor.transform(X_test).astype(np.float32)

y_train = y_train.to_numpy()
y_val = y_val.to_numpy()
y_test = y_test.to_numpy()

# 2. Define MLP
def build_mlp(width=32):
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(X_train.shape[1],)),
        tf.keras.layers.Dense(width, activation="relu"),
        tf.keras.layers.Dense(width // 2, activation="relu"),
        tf.keras.layers.Dense(1, activation="sigmoid")
    ])

    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )
    return model
```

```
# 3. Train the model
tf.keras.utils.set_random_seed(42)
model = build_mlp(width=32)

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=60,
    batch_size=32,
    verbose=0
)

# Plot loss and accuracy
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(history.history["loss"], label="Training")
axes[0].plot(history.history["val_loss"], label="Validation")
axes[0].set_title("Loss vs Epochs")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Binary Cross-Entropy")
axes[0].legend()
axes[0].grid(True)

axes[1].plot(history.history["accuracy"], label="Training")
axes[1].plot(history.history["val_accuracy"], label="Validation")
axes[1].set_title("Accuracy vs Epochs")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Accuracy")
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()

# 4. Evaluate on the held-out test set
probabilities = model.predict(X_test, verbose=0).ravel()
predictions = (probabilities >= 0.5).astype(int)

print("\nTest Accuracy:", accuracy_score(y_test, predictions))
print("Precision:", precision_score(
```

```
    y_test, predictions, zero_division=0
))
print("Recall:", recall_score(
    y_test, predictions, zero_division=0
))
print("F1 Score:", f1_score(
    y_test, predictions, zero_division=0
))

cm = confusion_matrix(y_test, predictions)
ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No Churn", "Churn"]
).plot()
plt.title("Customer Churn Confusion Matrix")
plt.show()

# 5. Compare hidden-layer widths
width_results = []

for width in [8, 16, 32, 64]:
    tf.keras.utils.set_random_seed(42)
    candidate = build_mlp(width=width)

    h = candidate.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=40,
        batch_size=32,
        verbose=0
    )

    loss, accuracy = candidate.evaluate(X_test, y_test, verbose=0)

    width_results.append({
        "Hidden width": width,
        "Test accuracy": accuracy,
        "Final validation loss": h.history["val_loss"][-1]
    })
```

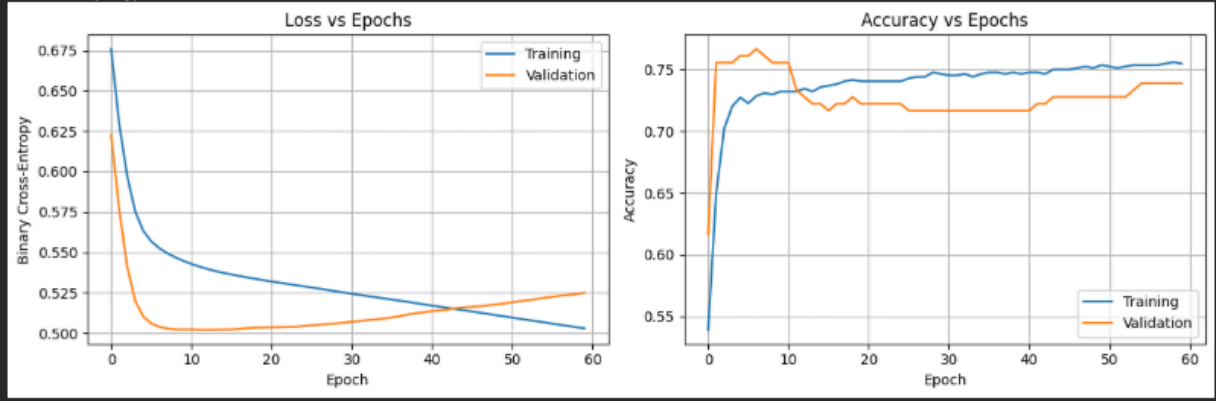
```
results_df = pd.DataFrame(width_results)
print("\nHidden-layer width comparison:")
print(results_df.round(4).to_string(index=False))

plt.figure(figsize=(7, 4))
plt.plot(results_df["Hidden width"],
         results_df["Test accuracy"], marker="o")
plt.xlabel("Hidden-layer width")
plt.ylabel("Test accuracy")
plt.title("Effect of Hidden-Layer Width")
plt.grid(True)
plt.show()
```

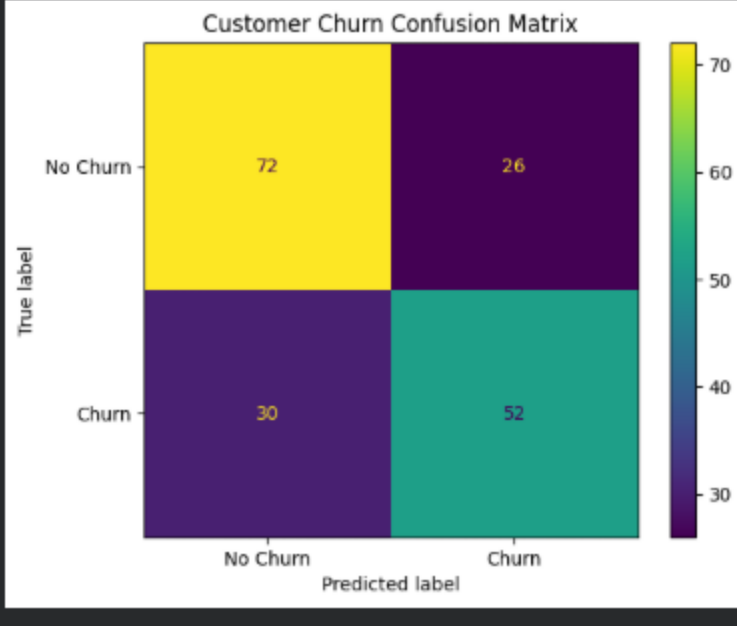
Output:

```
...    tenure  monthly_charges        contract  complaints  data_usage  Churn
0      52        47.168697  Month-to-month           3   19.975461      1
1      15        59.828738  Month-to-month           0   47.979891      1
2      72        29.169934  Month-to-month           0   36.787228      0
3      61        53.637046  Month-to-month           1   36.513975      0
4      21        72.251919        Two year           1   53.397477      0

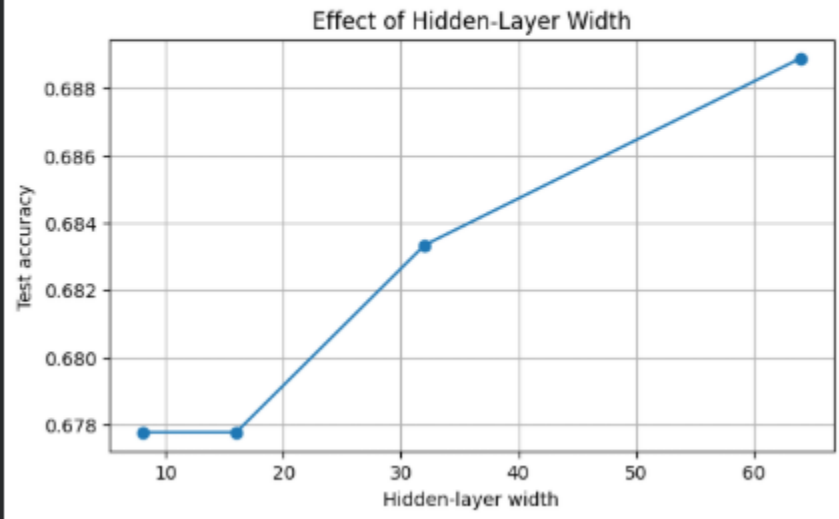
Churn distribution:
 Churn
0    657
1    543
Name: count, dtype: int64
```



```
Test Accuracy: 0.6888888888888889
Precision: 0.6666666666666666
Recall: 0.6341463414634146
F1 Score: 0.65
```




```
Hidden-layer width comparison:
 Hidden width  Test accuracy  Final validation loss
            8         0.6778                 0.5098
           16         0.6778                 0.4944
           32         0.6833                 0.5130
           64         0.6889                 0.5092
```



Experiment 10: L2, Dropout and Early Stopping Comparison

Problem Statement: A medical-records team trained a large MLP that scores 99% on training data but performs poorly on new patients. A small tabular dataset (around 500 records) for predicting diabetes onset from clinical features. The objective is to compare L2

regularisation, dropout and early stopping as remedies for overfitting. Use TensorFlow, Keras, NumPy, Matplotlib.

Tasks:

1. Train a deliberately large baseline MLP and plot its overfitting.
2. Retrain with L2 regularisation at three λ values.
3. Retrain with dropout at rates 0.2, 0.3 and 0.5.
4. Retrain with early stopping using validation loss.
5. Tabulate test accuracy and train–test gap for every model.

Formulas:

L2 regularization adds a penalty to large weights:

$$L_{total} = L_{data} + \lambda \sum_{i} w_i^2$$

Dropout randomly disables a fraction of hidden units during training to reduce dependence on particular neurons.

Early stopping monitors validation loss and stops training when performance no longer improves.

The train–test accuracy gap is

$$Gap = Trainingaccuracy - Testaccuracy$$

A large positive gap can indicate overfitting, though it should be interpreted alongside the absolute training and test scores.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
```

```
np.random.seed(42)
tf.keras.utils.set_random_seed(42)

# 1. Generate 500 synthetic clinical-style records
n = 500
X = np.random.normal(size=(n, 8))

# Create an illustrative relationship between features and outcome
logit = (
    0.9 * X[:, 0]
    + 0.7 * X[:, 1]
    - 0.5 * X[:, 2]
    + 0.6 * X[:, 3]
    + 0.5 * X[:, 0] * X[:, 1]
    + np.random.normal(0, 0.8, n)
)

prob = 1 / (1 + np.exp(-logit))
y = (np.random.random(n) < prob).astype(int)

# Training, validation, and test splits
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.40, random_state=42, stratify=y
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50,
    random_state=42, stratify=y_temp
)

# Fit scaling on training data only
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train).astype(np.float32)
X_val = scaler.transform(X_val).astype(np.float32)
X_test = scaler.transform(X_test).astype(np.float32)

# 2. Model builder
def build_model(l2_lambda=0.0, dropout_rate=0.0):
    regularizer = (
        tf.keras.regularizers.L2(l2_lambda)
```

```
        if l2_lambda > 0 else None
    )

    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(X_train.shape[1],)),
        tf.keras.layers.Dense(
            256, activation="relu",
            kernel_regularizer=regularizer
        ),
        tf.keras.layers.Dense(
            128, activation="relu",
            kernel_regularizer=regularizer
        ),
        tf.keras.layers.Dropout(dropout_rate),
        tf.keras.layers.Dense(
            64, activation="relu",
            kernel_regularizer=regularizer
        ),
        tf.keras.layers.Dense(1, activation="sigmoid")
    ])

    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )
    return model

results = []
histories = {}

def train_and_evaluate(name, l2_lambda=0.0,
                       dropout_rate=0.0,
                       early_stopping=False):
    tf.keras.backend.clear_session()
    tf.keras.utils.set_random_seed(42)

    model = build_model(l2_lambda, dropout_rate)

    callbacks = []
```

```
    if early_stopping:
        callbacks.append(
            tf.keras.callbacks.EarlyStopping(
                monitor="val_loss",
                patience=10,
                restore_best_weights=True
            )
        )

    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=100,
        batch_size=16,
        verbose=0,
        callbacks=callbacks
    )

    # Evaluate on train and test sets
    train_prob = model.predict(X_train, verbose=0).ravel()
    test_prob = model.predict(X_test, verbose=0).ravel()

    train_pred = (train_prob >= 0.5).astype(int)
    test_pred = (test_prob >= 0.5).astype(int)

    train_acc = accuracy_score(y_train, train_pred)
    test_acc = accuracy_score(y_test, test_pred)

    results.append({
        "Model": name,
        "Epochs": len(history.history["loss"]),
        "Train accuracy": train_acc,
        "Test accuracy": test_acc,
        "Train-test gap": train_acc - test_acc
    })

    histories[name] = history.history
    return model

# 3. Baseline: large network without regularization
```

```
baseline = train_and_evaluate("Baseline")

# 4. L2 regularization at three lambda values
for lam in [0.0001, 0.001, 0.01]:
    train_and_evaluate(f"L2 lambda={lam}", l2_lambda=lam)

# 5. Dropout at three rates
for rate in [0.2, 0.3, 0.5]:
    train_and_evaluate(f"Dropout={rate}", dropout_rate=rate)

# 6. Early stopping
train_and_evaluate("Early stopping", early_stopping=True)

# 7. Compare all models
results_df = pd.DataFrame(results)

print("\nModel comparison:")
print(results_df.round(4).to_string(index=False))

# Plot training and validation loss for baseline
plt.figure(figsize=(8, 5))
plt.plot(histories["Baseline"]["loss"], label="Training loss")
plt.plot(histories["Baseline"]["val_loss"], label="Validation loss")
plt.xlabel("Epoch")
plt.ylabel("Binary cross-entropy")
plt.title("Baseline MLP: Training vs Validation Loss")
plt.legend()
plt.grid(True)
plt.show()

# Plot validation loss for every model
plt.figure(figsize=(10, 6))
for name, history in histories.items():
    plt.plot(history["val_loss"], label=name)

plt.xlabel("Epoch")
plt.ylabel("Validation loss")
plt.title("Regularization Comparison")
plt.legend(fontsize=8)
plt.grid(True)
```

```
plt.tight_layout()
plt.show()

# Plot test accuracy and train-test gap
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

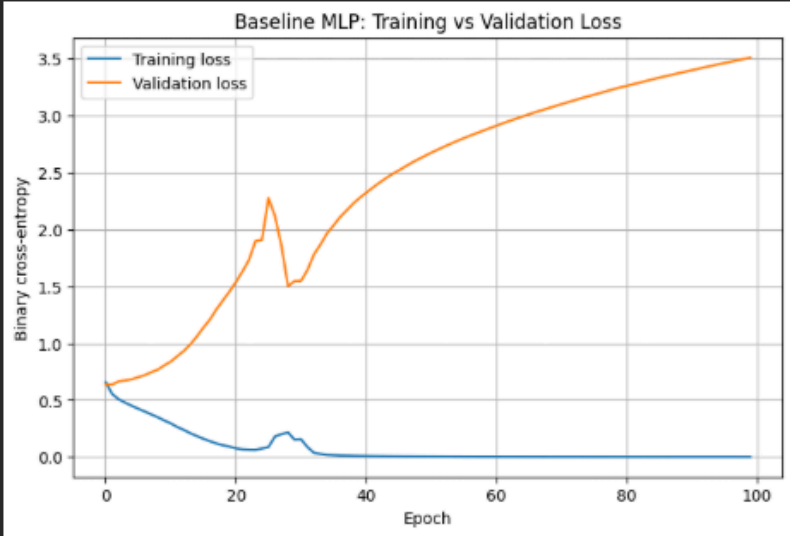
axes[0].bar(results_df["Model"], results_df["Test accuracy"])
axes[0].set_title("Test Accuracy by Model")
axes[0].set_ylabel("Accuracy")
axes[0].tick_params(axis="x", rotation=75)

axes[1].bar(results_df["Model"], results_df["Train-test gap"])
axes[1].set_title("Training-Test Accuracy Gap")
axes[1].set_ylabel("Accuracy gap")
axes[1].tick_params(axis="x", rotation=75)

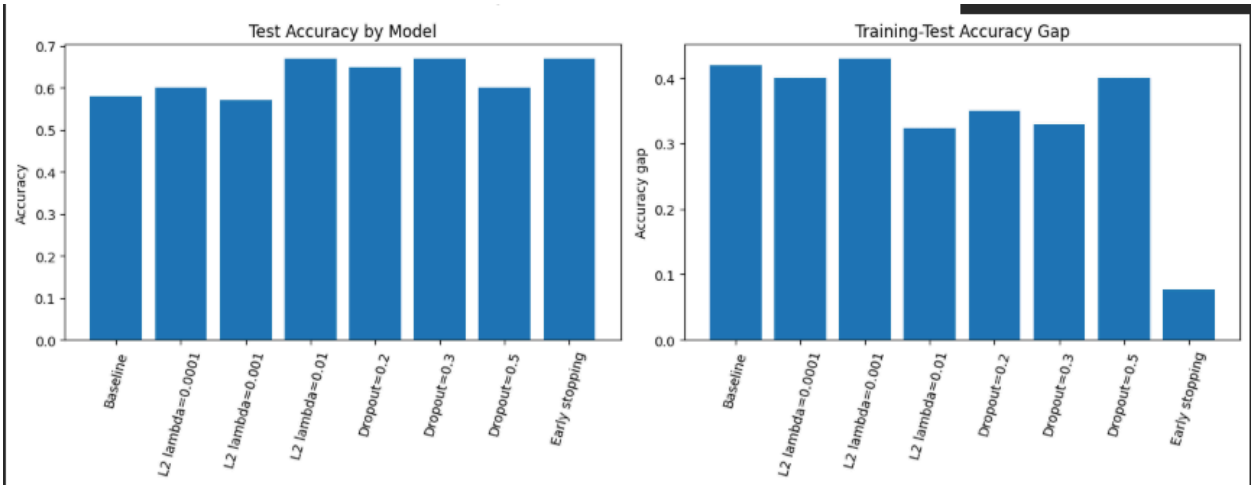
plt.tight_layout()
plt.show()
```

Output:

```
Model comparison:
                Model  Epochs  Train accuracy  Test accuracy  Train-test gap
...          Baseline     100          1.0000           0.58          0.4200
    L2 lambda=0.0001     100          1.0000           0.60          0.4000
     L2 lambda=0.001     100          1.0000           0.57          0.4300
      L2 lambda=0.01     100          0.9933           0.67          0.3233
        Dropout=0.2      100          1.0000           0.65          0.3500
        Dropout=0.3      100          1.0000           0.67          0.3300
        Dropout=0.5      100          1.0000           0.60          0.4000
     Early stopping       12          0.7467           0.67          0.0767
```




Regularization Comparison
Baseline
L2 lambda=0.0001
L2 lambda=0.001
L2 lambda=0.01
Dropout=0.2
Dropout=0.3
Dropout=0.5
Early stopping
Validation loss
Epoch
0.5
1.0
1.5
2.0
2.5
3.0
3.5
0
20
40
60
80
100


Test Accuracy by Model
Accuracy
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
Baseline
L2 lambda=0.0001
L2 lambda=0.001
L2 lambda=0.01
Dropout=0.2
Dropout=0.3
Dropout=0.5
Early stopping
Training-Test Accuracy Gap
Accuracy gap
0.0
0.1
0.2
0.3
0.4
Baseline
L2 lambda=0.0001
L2 lambda=0.001
L2 lambda=0.01
Dropout=0.2
Dropout=0.3
Dropout=0.5
Early stopping